Group X Progress Report:
My Group's Project Name

Aditya Tripuraneni, Vanessa Lai, Third Author
{tripuraa,laiv3,macid3}@mcmaster.ca

1 Introduction

Distributed systems record their activity in logs, and engineers rely on these logs to find out when something has gone wrong. The volume of logsmakes manual inspection impractical: the HDFS cluster in our dataset produced over 11 million log lines in under two days, roughly 290,000 lines per hour [1, 2]. Failures are also rare, affecting only about 3% of sessions, so they are very hidden in the data. Supervised models can learn to recognize these failures, but they need labeled examples, and labeling logs at this scale takes time and money. This raises the question of whether labels are necessary at all. Our main research question is whether unsupervised models, which train without needing labels, can detect anomalies as well as supervised models. We will compare two supervised classifiers (logistic regression and XGBoost) with two unsupervised detectors (Isolation Forest and a DeepLog-style LSTM) using the same preprocessing and the same train/validation/test split.

2 Related Work

Here, talk about the related work you encountered for your approach. Cite at least 5 references. Refer to item 2. No one has done exactly your task? Write about the most similar thing you can find. This should be around 0.25-0.5 pages.

3 Dataset

3.1 Source and Collection

We use the HDFS_v1 dataset from Loghub [1, 3]. Its logs were generated by running benchmark workloads on a 203-node Hadoop Distributed File System (HDFS) cluster [1, 4] and contain 11,175,629 log lines, about 1.5 GB [2]. HDFS splits files into large blocks, replicates each block on multiple DataNodes (three by default), and records block locations on a NameNode [5]. In the data, every log line contains a block ID, and most logged block sizes are 64 MB (94% are 67,108,864 bytes). Grouping the lines by block ID yields one *session* per block [1, 2].

Sessions (block IDs)	575,061
Anomalous sessions	16,838 (2.93%)
Event types (of 30 templates)	29
Length: median / 99th pct / max	19 / 33 / 298

Table 1: Summary of the HDFS_v1 sessions. Counts agree with [2]; length statistics are our own computation.

3.2 Properties of the Data

One data point is one block session: the time-ordered sequence of log events for a single block, with a binary normal/anomalous label. Loghub defines 30 event templates (E1–E30), message types in which variable parts such as block IDs and IP addresses are replaced by wildcards [3]. Only 29 occur in the sessions, in agreement with [2]. Reading the template names, a typical normal session follows a regular lifecycle: a block is allocated (E22), received by three DataNodes (E5, E11, E9), recorded by the NameNode (E26) once per replica, then deleted (E23, E21), so the same events repeat three times, once per replica [5]. Anomalous sessions break this pattern. For example, `[E5, E5, E22, E7, E11, E10, E14, E7]` contains three exception events (E7, E10, E14) and never reaches E26.

Table 1 summarizes the data. It is highly imbalanced: 16,838 of the 575,061 sessions are anomalous [2], i.e., 2.93%, and we reproduced these counts from the Loghub files. Sessions are short (median 19 events), and length does not separate the classes (median 19 for normal, 20 for anomalous). We found no missing values and no duplicate block IDs.

3.3 Preprocessing

Our proposal planned to parse the raw logs into event templates with Drain [6]. We found that Loghub already provides parsed event templates and per-block event sequences, so we use those directly. The remaining conversion is handled by our own loader functions, which keeps the pipeline simple. The loader (1) joins each block's event sequence with its label by block ID, checking the join is exactly one-to-one; (2) converts each stored sequence string into a list of integer event IDs; (3) drops the `Type`, `Latency`, and `TimeInterval` columns, where `Type` holds the anomaly type that Loghub provides alongside the labels [1] and is populated for exactly the anomalous sessions, so using it would leak the label, and the other two fall outside our model input (the event sequence alone); and (4) encodes labels as 0 (normal) and 1 (anomalous).

3.4 Train, Validation, and Test Splits

Our dataset has a 70/10/20 train/validation/test split. We stratified the dataset on the binary label so each split keeps the same 2.93% anomaly rate. Without stratification, a random split could put too few of the rare anomalies into validation or test, and the four models would no longer be compared on the same class balance.

3.5 Annotation

We did not annotate any data. The labels were assigned by the dataset's authors, who manually labeled each block session as normal or anomalous using handcrafted rules [1]. We treat them as ground truth.

4 Features

Describe any features you used for your model, or how your data was input to your model. Are you doing feature engineering or feature selection? Are you learning embeddings? Is it all part of one neural network? Refer to item 2. This may range from 0.25 pages to 0.5 pages.

5 Implementation

Describe your model and implementation here. Refer to item 4. This may take around a page.

6 Results and Evaluation

How are you evaluating your model? What results do you have so far? What are your baselines? Refer to item 5. This may take around 0.5 pages.

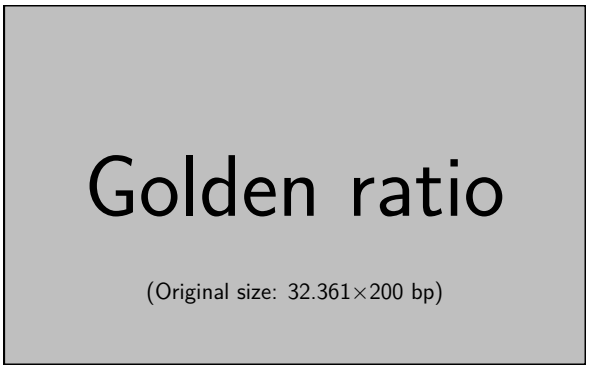


Figure 1: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

7 Feedback and Plans

We plan to use 5-fold cross-validation over the training and validation data, keeping the test set fixed, to measure how much each model's scores and chosen thresholds vary across folds.

7.1 Tables and figures

See Table 2 for an example of a table and its caption. See Figure 1 for an example of a figure and its caption.

7.2 Citations

Table 2 shows the syntax supported by the style files. We encourage you to use the natbib styles. You can use the command `\citet` (cite in text) to get "author (year)" citations, like this citation to a paper by **(author?)** [7]. You can use the command `\citep` (cite in parentheses) to get "(author, year)" citations [7]. You can use the command `\citealp` (alternative cite without parentheses) to get "author, year" citations, which is useful for using citations within parentheses (e.g. 7).

7.3 References

Many websites where you can find academic papers also allow you to export a bib file for citation or bib formatted entry. Copy this into the `custom.bib` and you will be able to cite the paper in the LaTeX. You can remove the example entries.

7.4 Equations

An example equation is shown below:

$$A = \pi r^2 \tag{1}$$

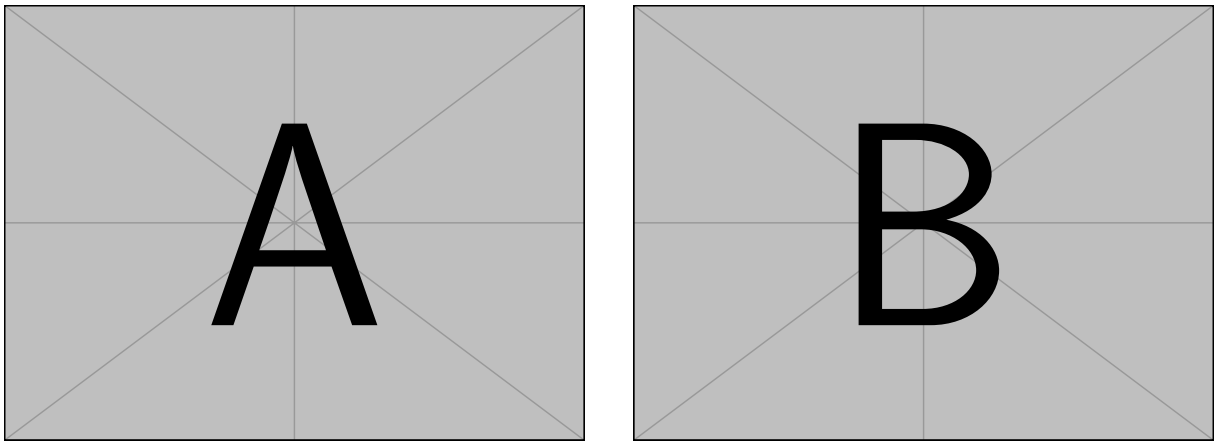


Figure 2: A minimal working example to demonstrate how to place two images side-by-side.

Output	natbib command	ACL only command
[7]	`\citep`	
7	`\citealp`	
(author?) [7]	`\citet`	
[**(year?)**]	`\citeyearpar`	
(author?)'s (**(year?)**)		`\citeposs`

Table 2: Citation commands supported by the style file.

Labels for equation numbers, sections, subsections, figures and tables are all defined with the `\label{label}` command and cross references to them are made with the `\ref{label}` command. This an example cross-reference to Equation 1. You can also write equations inline, like this: $A = \pi r^2$.

Team Contributions

Write in this section a few sentences describing the contributions of each team member. What did each member work on? Refer to item 7.

References

[1] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," arXiv:2008.06448, 2020.

[2] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE 27th Int. Symp. Software Reliability Engineering (ISSRE)*, 2016, pp. 207–218.

[3] LogPAI, "Loghub: System log datasets," https://github.com/logpai/loghub, accessed: Nov. 2, 2026.

[4] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. 22nd ACM Symp. Operating Systems Principles (SOSP)*, 2009, pp. 117–132.

[5] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[6] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[7] D. Gusfield, *Algorithms on Strings, Trees and Sequences*. Cambridge, UK: Cambridge University Press, 1997.

[8] R. K. Ando and T. Zhang, "A framework for learning predictive structures from multiple tasks and unlabeled data," *Journal of Machine Learning Research*, vol. 6, pp. 1817–1853, Dec. 2005.

[9] G. Andrew and J. Gao, "Scalable training of L1-regularized log-linear models," in *Proceedings of the 24th International Conference on Machine Learning*, 2007, pp. 33–40.

[10] M. S. Rasooli and J. R. Tetreault, "Yara parser: A fast and accurate dependency parser," *Computing Research Repository*, vol. arXiv:1503.06733, 2015, version 2. [Online]. Available: http://arxiv.org/abs/1503.06733